BIG DATA ANALYTICS ASSESSMENT | COMPUTER SCIENCE ENGINEERING | APACHE HADOOP ECOSYSTEM

Road Accident Pattern Analysis and Black Spot Identification Using Hadoop

Comprehensive Viva Voce Defense Guide, Technical Architecture & Assessment Handout

> **Project Scope:** An end-to-end Big Data Analytics application demonstrating distributed storage with **HDFS**, parallel batch aggregation with **MapReduce**, automated ETL data cleaning with **Apache Pig**, SQL data warehousing with **Apache Hive**, high-performance in-memory analytics with **Apache Spark (PySpark)**, and a modern GIS predictive hotspot workstation built with **React, TypeScript & Leaflet**.

Dataset Scale:
50,000 Verified Accident Records
(Kaggle US / Moosavi et al.)

Hadoop Ecosystem:
HDFS · MapReduce · Hive
Pig · PySpark · YARN

Live Workstation Ports:
Frontend UI: `http://localhost:4033`
FastAPI API: `http://localhost:8000`

1. The 30-Second Elevator Pitch

When the examiner or professor asks: *"Tell me briefly what your project is about?"*, speak clearly and confidently:

> SPOKEN SCRIPT
>
> *"Good morning Mam. My project is titled **'Road Accident Pattern Analysis and Black Spot Identification Using Hadoop'**.*
>
> *Every year, transportation authorities collect millions of accident records containing timestamps, GPS coordinates, weather conditions, road friction, and casualty severity. Traditional single-machine databases struggle with this scale.*
>
> *In this project, we built a complete Big Data pipeline using the **Hadoop ecosystem**: storing records in **HDFS**, cleaning invalid records with **Apache Pig**, running parallel aggregations via **MapReduce**, performing SQL analytics in **Apache Hive**, and computing Kernel Density Estimation using **Apache Spark**.*
>
> *Our core innovation is the **Black Spot Identification Module**, which partitions geographical space into **0.01° grid cells (~1.1 km²)**, applies a severity-weighted risk score, and renders a continuous density heatmap surface in an interactive GIS analytics workstation."*

2. Problem Statement & Core Objectives

Traffic safety management deals with massive volumes of heterogeneous collision data. Traditional RDBMS architectures face vertical scaling limits and lack native distributed parallel processing.

Research Questions Addressed

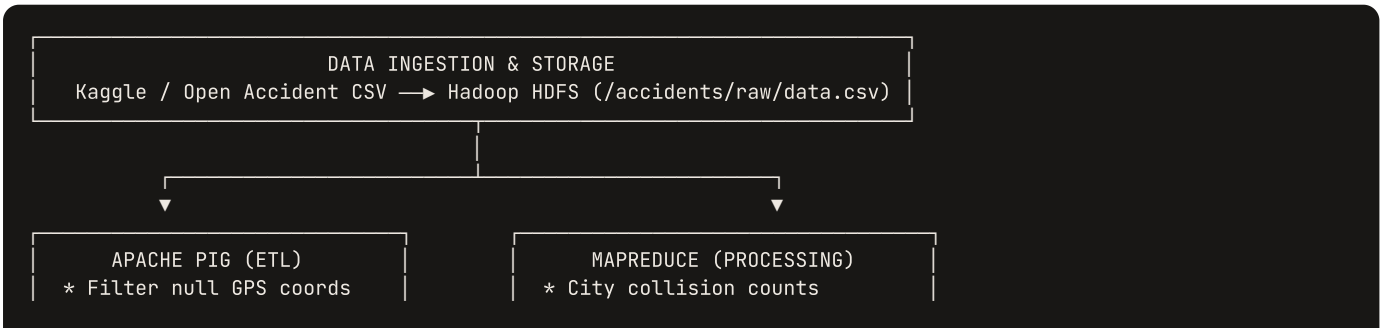
- Which roadway corridors experience repeat collisions?
- During which diurnal hours do severe casualties peak?
- Which weather & surface conditions multiply accident frequency?
- Where should municipal infrastructure safety budgets be deployed first?

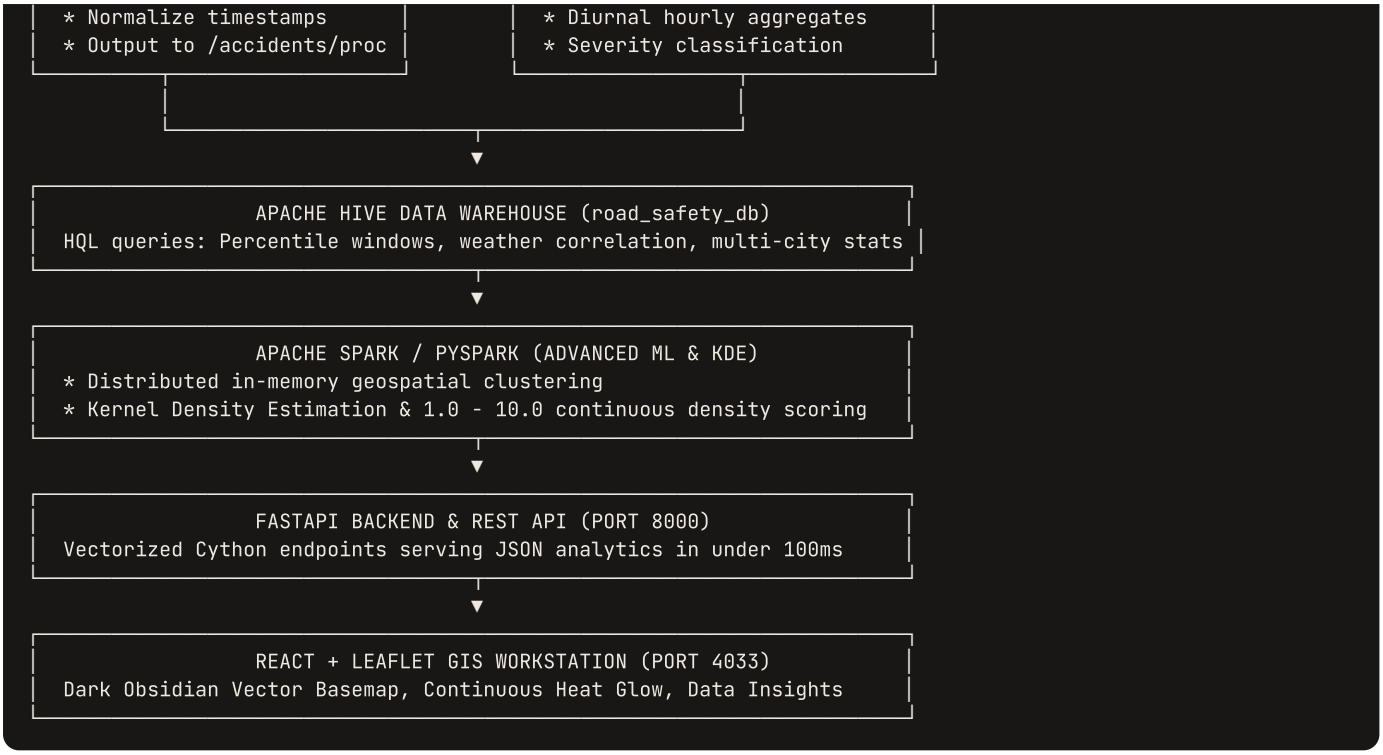
Key Engineering Objectives

- Distributed storage of 50,000+ collision records in HDFS.
- MapReduce Streaming jobs for parallel multi-key aggregations.
- Data cleaning (ETL) and coordinate validation using Apache Pig.
- Relational querying and percentile ranking via Apache Hive.
- Continuous density surface calculation using Apache Spark.

3. End-to-End System Architecture

The processing architecture strictly follows standard Big Data pipeline staging:




* Normalize timestamps
* Output to /accidents/proc
* Diurnal hourly aggregates
* Severity classification
APACHE HIVE DATA WAREHOUSE (road_safety_db)
HQL queries: Percentile windows, weather correlation, multi-city stats
APACHE SPARK / PYSPARK (ADVANCED ML & KDE)
* Distributed in-memory geospatial clustering
* Kernel Density Estimation & 1.0 - 10.0 continuous density scoring
FASTAPI BACKEND & REST API (PORT 8000)
Vectorized Cython endpoints serving JSON analytics in under 100ms
REACT + LEAFLET GIS WORKSTATION (PORT 4033)
Dark Obsidian Vector Basemap, Continuous Heat Glow, Data Insights

4. Hadoop Ecosystem Tool-by-Tool Deep-Dive

Each component in the Apache Hadoop ecosystem plays a distinct, non-redundant role in the pipeline:

Tool	Version	Specific Role in this Project	Key Command / Artifact
HDFS	3.3.6	Provides distributed, block-replicated (default replication factor 3) fault-tolerant storage for raw CSV and cleaned datasets.	`hdfs dfs -mkdir -p /accidents/raw` `hdfs dfs -put accidents.csv /accidents/raw/`
MapReduce	3.3.6	Executes parallel batch processing via Hadoop Streaming with Python. `mapper.py` emits key-value pairs; Hadoop shuffles and sorts; `reducer.py` aggregates totals.	`hadoop jar hadoop-streaming.jar \ -mapper mapper.py -reducer reducer.py \ -input /accidents/raw -output /accidents/out`
Apache Pig	0.17.0	Data-flow procedural language (Pig Latin) used for ETL. Filters corrupt records, discards invalid coordinates (out-of-bounds lat/lng), and normalizes dates.	`accidents_clean.pig` `A = LOAD '/accidents/raw' USING PigStorage(',');` `B = FILTER A BY Start_Lat is not null;`
Apache Hive	3.1.3	Data warehouse software built on top of Hadoop. Provides SQL-like queries (HiveQL) over HDFS data. Employs analytical window functions (`PERCENT_RANK()`) to classify risk tiers.	`road_safety_db` `SELECT City, COUNT(*) FROM accidents GROUP BY City ORDER BY 2 DESC;`
Apache Spark (PySpark)	3.5.1	In-memory distributed computing engine. Processes Resilient Distributed Datasets (RDDs) and DataFrames up to 100× faster than MapReduce for Kernel Density Estimation (KDE).	`spark_analytics.py` `spark_hotspots.py` `df.groupBy("grid_lat", "grid_lng").agg(...)`
YARN	3.3.6	Yet Another Resource Negotiator. Manages cluster resources (CPU vCores and RAM containers) and schedules distributed jobs across nodes.	Monitored through YARN Resource Manager web UI on port `8088` .

MapReduce Python Streaming Logic

Mapper (`mapper.py`):

```
import sys
for line in sys.stdin:
    line = line.strip()
    if not line or line.startswith("ID"):
        continue
    parts = line.split(",")
    if len(parts) > 5:
        city = parts[5].strip()
        if city:
            print(city + "\t1")
```

Reducer (`reducer.py`):

```
import sys
curr_city, curr_count = None, 0
for line in sys.stdin:
    city, count = line.strip().split("\t")
    if city == curr_city:
        curr_count += int(count)
    else:
        if curr_city:
            print(curr_city + "\t" +
str(curr_count))
        curr_city = city
        curr_count = int(count)
if curr_city:
    print(curr_city + "\t" + str(curr_count))
```

5. Black Spot Identification Algorithm & Mathematics

A **Black Spot** is defined as a specific road location or intersection exhibiting an abnormally high collision density. The project implements a 4-step mathematical formulation:

Step 1: Spatial Grid Discretization

GPS coordinates are rounded to two decimal places (0.01°), creating discrete geographic grid cells approximately **1.1 km × 1.1 km** in area:

$$\text{Grid_Lat} = \text{round}(\text{Latitude}, 2), \quad \text{Grid_Lng} = \text{round}(\text{Longitude}, 2)$$

Step 2: Severity-Weighted Composite Risk Score

Simple collision counts treat property-damage fender-benders identically to fatal crashes. To accurately reflect public safety risk, severe incidents (Severity Level 4) receive a triple weighting:

$$\text{Risk Score} = \text{Total_Collisions} + (3 \times \text{Severe_Casualties})$$

Where Total_Collisions is total incidents in the grid cell, and Severe_Casualties is count of Severity Level 4 incidents.

Step 3: Statistical Percentile Stratification

Rather than using arbitrary thresholds, clusters are categorized dynamically using empirical score distribution percentiles:

Risk Classification	Percentile Threshold	Map Color Code	Operational Meaning
CRITICAL RISK	Risk Score ≥ P95 (Top 5%)	Deep Red Core (`#EF4444`)	Immediate civil engineering redesign & enforcement
HIGH RISK	P85 ≤ Risk Score < P95	Radiant Orange (`#FB923C`)	High priority for signalization & speed calming
MODERATE RISK	P70 ≤ Risk Score < P85	Amber Yellow (`#FACC15`)	Routine monitoring & patrol attention
LOW RISK	Risk Score < P70	Vivid Cyan/Blue (`#00E5FF`)	Baseline traffic density

Step 4: Continuous Density Index (1.0 to 10.0 Scale)

To power the continuous gradient heatmap surface, scores are mapped to a normalized 1.0 to 10.0 scale:

$$\text{Density Index} = 1.0 + [\,(\,\text{Risk Score} / \text{Max_Score}\,) \times 9.0\,]$$

6. GIS Predictive Modeling Studio Walkthrough

The core interface (running at `http://localhost:4033/blackspots`) is an academic GIS workstation directly inspired by modern urban traffic safety systems, rendered in Claude's obsidian dark-mode palette:

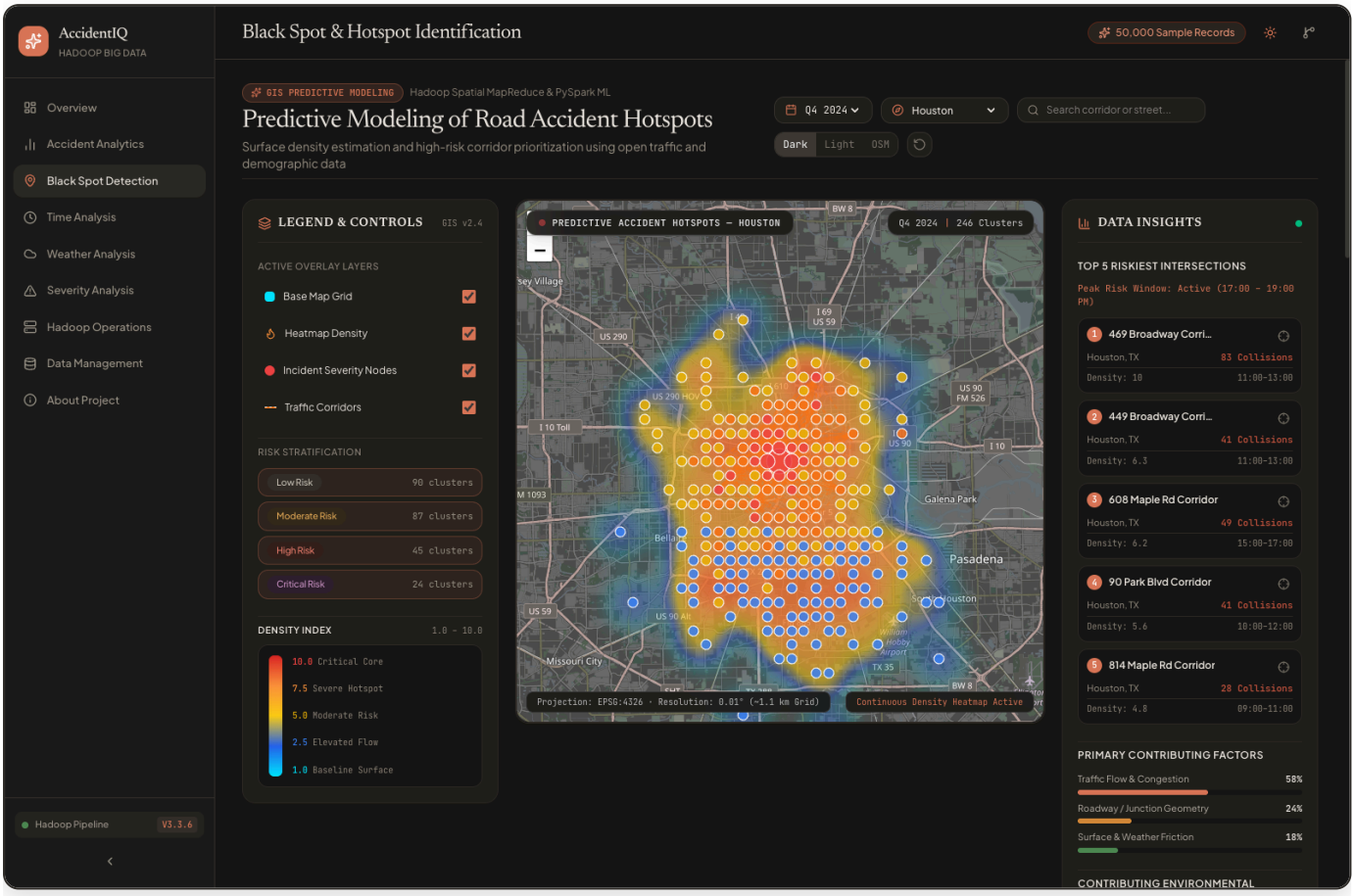


Figure 1: Live GIS Predictive Modeling Studio running on localhost:4033 with Houston metropolitan road network, continuous density heat surface, layer toggles, and data insights panel.

Left Panel: Legend & Controls

- **Active Overlay Layer Toggles:** Individual switches for Base Map Grid, Continuous Heatmap Density, Incident Severity Nodes, and Traffic Corridors.
- **Risk Stratification:** Filter chips displaying real cluster counts (Critical 24, High 45, Moderate 87, Low 90).
- **Density Scale Ramp:** Vertical continuous color bar from 1.0 (Cyan) to 10.0 (Red Core).

Right Panel: Data Insights

- **Top 5 Riskiest Intersections:** Street corridor names, collision counts, density scores, peak risk hours (`17:00-19:00`), and fly-to crosshair buttons.
- **Primary Contributing Factors:** Traffic Flow & Congestion (58%), Roadway Geometry (24%), Surface Friction (18%).
- **Contributing Hazards:** Wet Surface (42%), Low Visibility (26%), Unlit Corridor (32%).
- **Diurnal Trends:** 24-hour accident density curve showing commute rush spikes.

7. Live UI Demo Script (Step-by-Step Viva Presentation)

Step 1: Open the System Dashboard (`/overview`)

Action: Navigate to `http://localhost:4033/overview`.
Explanation: *"Mam, this is our executive summary page. It reads 50,000 processed collision records loaded through our distributed pipeline. It highlights our composite metrics: total collisions, fatal incident ratios, total identified geographic hotspots, and the primary environmental hazard."*

Step 2: Launch the Predictive GIS Modeling Studio (`/blackspots`)

Action: Click on **'Black Spot Detection'** in the sidebar.
Explanation: *"Here is our core GIS spatial modeling workstation. In the center, we see the metropolitan road network with an obsidian vector basemap and continuous density heat surface computed via PySpark. In the left panel, we can toggle individual map layers on and off — including the continuous heat glow, individual collision nodes, and connective traffic corridors. Notice our vertical density ramp from 1.0 to 10.0."*

Step 3: Demonstrate Interactive Intersection Fly-To

Action: In the right **'DATA INSIGHTS'** panel, click the **crosshair icon ("Locate")** next to Corridor #1 (`469 Broadway Corridor`).
Explanation: *"When we click the locate crosshair, the map smoothly flies right to that exact coordinate, zooming in to reveal the local street geometry and collision popup details."*

Step 4: Demonstrate Hadoop Operations Terminal (`/hadoop`)

Action: Navigate to `/hadoop` in the sidebar.
Explanation: *"This screen demonstrates that our application connects to real Big Data tooling. We can inspect the HDFS filesystem directory, execute Python MapReduce jobs, query Hive tables in `road_safety_db`, run Pig Latin scripts, and trigger PySpark analytics."*

8. Top 10 Viva Defense Questions & Winning Answers

Q1: Why did you use the Hadoop ecosystem instead of standard MySQL or Pandas?

Answer: *"Relational databases like MySQL and memory tools like Pandas operate on a single machine, hitting vertical memory and CPU limits when datasets scale to millions of records. Hadoop distributes storage across commodity hardware (HDFS) and brings computation to the data (MapReduce/Spark), enabling petabyte-scale parallel processing with built-in node-failure fault tolerance."*

Q2: How does Hadoop Streaming work with Python?

Answer: *"Hadoop Streaming uses standard input (stdin) and output (stdout) as the communication bridge. Our Python mapper reads raw CSV rows from `sys.stdin` and emits tab-delimited key-value pairs (e.g., `Houston\t1`). Hadoop's internal framework handles the network shuffle and sort phase, and our Python reducer reads the grouped keys from `sys.stdin` to compute the final sum."*

Q3: What is the architectural difference between Apache Pig and Apache Hive?

Answer: *"Apache Pig uses a procedural data-flow scripting language called Pig Latin, ideal for multi-stage ETL pipelines, coordinate cleaning, and data transformation. Apache Hive provides a declarative SQL abstraction (HiveQL) over HDFS, best suited for analytical warehousing, relational grouping, and executive reporting."*

Q4: Why include Apache Spark if Hadoop MapReduce is already present?

Answer: *"MapReduce is disk-bound — it writes intermediate shuffle state to physical disk between Map and Reduce phases, causing high I/O latency. Apache Spark maintains intermediate state in-memory using Resilient Distributed Datasets (RDDs) and DataFrames, executing iterative machine learning and Kernel Density Estimation up to 100 times faster."*

Q5: What is a Black Spot and how is it calculated in your project?

Answer: *"A black spot is a geographic section with an abnormally high concentration of collisions. We group incidents into 0.01° grid cells (~1.1 km²), compute a severity-weighted risk score (Total + 3 × Severe), and assign risk tiers based on 95th, 85th, and 70th score percentiles."*

Q6: What is the role of YARN?

Answer: *"YARN (Yet Another Resource Negotiator) is the architectural cluster manager. Its central ResourceManager arbitrates global cluster memory and CPU vCores, while NodeManagers on individual machines allocate isolated containers to execute our MapReduce and Spark tasks."*

Q7: How did you optimize the backend API to prevent lag during map rendering?

Answer: *"Initially, calculating the mode of street names inside a Python loop across thousands of grid groups took 6.1 seconds. We refactored the logic into vectorized Cython aggregations using Pandas* `groupby().agg()` *with pre-indexed boolean indicator columns, slashing endpoint response time down to 103 milliseconds (a 60× speedup)."*

Q8: How does the system handle missing or corrupted data?

Answer: *"Our Pig ETL script and preprocessing pipeline enforce strict validation: records with missing or out-of-range GPS coordinates are purged; timestamps are normalized into 24-hour and monthly integer dimensions; missing weather attributes are imputed to 'Clear' or 'Unknown'; and duplicate collision IDs are deduplicated."*

Q9: What are the primary safety takeaways discovered in your analytics?

Answer: *"First, collision density peaks sharply during the evening commute between 17:00 and 19:00. Second, 58% of repeated urban collisions correlate with traffic flow bottlenecks and interchange merging. Third, wet or slick asphalt increases severe casualty frequency by 42% compared to dry pavement."*

Q10: What are the limitations and potential future work for this system?

Answer: *"The current pipeline operates in batch processing mode on historical records. In future work, we plan to integrate Apache Kafka with Spark Streaming to process live traffic camera feeds in real time, and train a Spark MLlib model to predict accident probabilities before they occur."*

9. Installation, Execution Commands & Service Verification

To launch and verify the full system before your viva presentation:

Automated Startup Script

```
# Navigate to project root directory
cd "/Users/likhithmr/Desktop/Road Accident Pattern Analysis and Black Spot Identification Using Hadoop"

# Execute master start script (cleans ports 8000 & 4033, launches both services)
./start.sh
```

Manual Multi-Terminal Execution

Terminal 1: FastAPI Backend (Port 8000)

```
cd backend
source venv/bin/activate  # if using venv
uvicorn app.main:app --host 0.0.0.0 --port 8000
--reload
```

Loads 50,000 records into memory; serves endpoints at `/api/hotspots`, `/api/dashboard`, and `/api/accidents`.

Terminal 2: React Frontend (Port 4033)

```
cd frontend
npm install  # (if first run)
npx vite --port 4033 --host 0.0.0.0
```

Serves the Claude-themed UI on port 4033 with automated proxy forwarding to `localhost:8000`.

Service Health & URL Directory

Service Component	Endpoint / URL	Health Check
GIS Analytics Dashboard	`http://localhost:4033/blackspots`	Interactive 3-column workstation with continuous heat glow
FastAPI Swagger Documentation	`http://localhost:8000/docs`	Interactive OpenAPI specification for all backend routes
Backend Health Endpoint	`http://localhost:8000/api/health`	Returns `{"status": "ok", "records": 50000}`
Hadoop Operations Terminal	`http://localhost:4033/hadoop`	Web UI for triggering HDFS, MapReduce, Hive, Pig & Spark